Promo Finder: Solution Doc (Problem 1, Find the Promos)

Priyansh | Roll no. 22410028 | Geological Technology, IIT Roorkee

1. Why I chose this problem

It comes with real data, and its output can be checked. A promo is an episode plus start and end times, so I can read the script lines at those times and see whether the pick holds up. It is also a repeating job, since a new show arrives every week. The tool only finds and ranks promos. A person still makes the final call, and there is no editing, music or text.

2. Assumptions

- Promos stay inside one episode. I left out cross-episode joins because they need story tracking across episodes, and a wrong join can confuse a new viewer or give away a later twist. Joining several moments inside one episode is supported.
- Text only. Hooks and cliffhangers are judged from the dialogue and action lines. I cannot see acting, visuals or sound, so a line that looks flat on paper may work on screen.
- A cut never lands in the middle of a spoken line, and I assume the screenplay timestamps are accurate.
- A hook is a line or action that starts in the first 3 seconds of the clip. A cliffhanger is the clip's last line, and it must not be a closing line such as "thank you" or "the end". Total length is 20 to 60 seconds.

3. How it works

Step	Done by	What it does
parse.py	Code	Turns each episode file into timed blocks (camera, action, sound, dialogue). On the pack: 50 episodes, 3,727 blocks.
score.py	LLM	One call per episode. It sees the timeline, the episode logline and the neighbouring loglines, and proposes up to 4 promos with hook, context, open-question and spoiler scores plus the hook and cliffhanger lines.
candidates.py	Code	Treats the model's timestamps as untrusted and moves them to the nearest legal cut within 4 seconds, keeping the 20 to 60 second limit.
validate.py	Code	Checks again: length, no mid-line cut, something happens in the first 3 seconds, the quoted hook and cliffhanger really are in the clip, no closing line at the end. Failures go to rejected.json with the reason.
rank.py	Code, 1 LLM call	Score = 0.35 hook + 0.40 open question + 0.25 context, minus a spoiler penalty, with at most 2 picks per episode. One LLM call then orders the shortlist and gives reasons for the top 5.

The model, base URL and key name are set in one config file, so another OpenAI-compatible model can be swapped in without changing code. Saved model replies are reused only when the prompt is identical, so a re-run costs nothing and a different show never picks up old replies. A small Streamlit app lets you upload new episode files, run the pipeline, read each pick next to its script lines, and label it good, ok or bad.

4. Results on the 50-episode pack

The model made 145 proposals. 46 passed validation and 15 became the final picks, spread over 13 episodes (20.9 to 59.7 seconds, 7 with more than one segment). 96 proposals were rejected: 58 because the total length fell outside 20 to 60 seconds (all of them multi-segment), 26 because they could not be moved to a legal cut, 10 because the claimed cliffhanger was not the clip's last line, and 7 because the claimed hook was not in the first 3 seconds (some had two reasons). The top 5:

#	Ep	Segments	Hook	Cliffhanger
1	1	00:11.3-00:41.2	You should turn around, Juliana.	Because he is marrying me.
2	5	00:06.8-00:41.5 + 00:42.5-00:58.2	You will not leave this church carrying the Ashcroft name.	(stage direction: the book hits the wet pavement)
3	12	00:00.5-00:27.9 + 00:28.1-00:35.2 + 00:44.0-01:09.2	Put out your hands, veteran. You are under arrest for theft.	Fail, or refuse, and your husband hangs from the gallows.
4	43	00:00.6-00:21.5	You're lying!	No. He is your Duke.
5	9	00:02.0-00:14.2 + 00:34.9-00:37.4 + 00:58.8-01:21.6	Stay back. I'll open it.	Gossip has a way of becoming dangerous.

I would post #1 first: a betrayal at the altar that a stranger understands at once, with no spoiler. #3 comes next because it opens a different storyline (an arrest) with a clear threat. Without that variety the top of the list would be mostly the wedding scene. Full list and reasons: outputs/promos.md.

5. Where it goes wrong

- I have no real measure of quality yet. Passing validation only means a pick is well formed and quotes the script correctly, not that viewers would stop scrolling. The ranking weights are my own guesses, and the real test is to A/B the picks on view-through and installs.
- The model gives itself top scores. All 15 picks have hook 5 and open question 5 (13 also have context 5), so the code score barely separates them and the order depends mostly on the single re-rank call.
- Some hooks and endings are weak. Episode 5 (#2) ends on a stage direction, not a line or a question. "Alexander!" (episodes 10 and 35) and "Look at him." (episode 4) are thin openers on paper. The re-rank reasons come from the episode logline, so one can describe something that is not in the clip.
- Multi-segment joins are the weakest part. 58 of the 96 rejections were joins whose total length broke after each segment was moved to a legal cut on its own. Single clips use a snap that respects the length limit and joins do not, which is fixable. A few joins are nearly continuous (gaps of 0.2 to 1 second).

- Later episodes give less. Episodes 35 to 50 produced 2 final picks and 10 rejections. I have not found out whether the model got worse or those episodes are weaker.
- Picks can repeat a storyline. The cap is per episode, so several picks can come from the same scene (episodes 1, 2, 4 and 5 are all the wedding).

6. Where a person stays in the loop

The tool shortlists and the editor decides. Each pick has a "needs a human look" flag for low context, spoiler risk or an unclear ending, but the flags fired on none of the 15 final picks, so for now the useful check is looking at the whole shortlist: 15 clips for a 50-episode show instead of watching every episode. The labels from that review go in outputs/review.csv and are the starting set for tuning.

7. Scale and cost

- One 50-episode show used about 77k input tokens (an estimate, characters divided by 4) and about 20k output tokens, in 51 model calls. At small-model prices that is a few cents per show. I assumed about $0.10 and $0.40 per million input and output tokens, which should be checked against current pricing. A week of new shows should cost well under a dollar.
- On the free tier it takes about 10 to 20 minutes per show, because of a 6 second pause between calls and retries on rate limits. Episodes are independent, so a paid key (pause set to 0) can run them much faster.
- It works unchanged for any show in the same screenplay format. Another format needs only a new parser. Next steps: length-aware snapping for joins, a rule against repeating a storyline, and a stronger model for the re-rank.

8. Tools used, and one place the AI got it wrong

I used Claude for design, code drafting and review, and Gemini Flash-Lite through the free API as the model the tool calls. Gemini often returned a hook or cliffhanger said to be copied exactly from the script when it was not in the clip: 17 of the 96 rejections (7 hook, 10 cliffhanger). That is why validate.py checks the quoted lines against the real script. A second miss turned up in my final review: the re-rank step returned ids as strings ("4"), my code accepted only integers, so the model's ordering was silently dropped while its reasons stayed attached to the wrong cards. I fixed it by converting the ids, and the ranking above is the corrected one.